Unit 2 Hands-on

Name: Bidisha Paul
SRN: PES2UG23CS131
6B CSE

Class Notes:

Part 1: Langchain Foundation

The `temperature` parameter, controls the randomness of the Language Model's output.

- **Lower temperature (e.g., 0.0):** Makes the model's output more deterministic and focused. It will tend to choose the most probable words, resulting in more consistent and less creative responses. As we saw with `llm_focused` (temperature=0.0), it produced the exact same definition for 'Idea' twice.
- **Higher temperature (e.g., 2.0):** Makes the model's output more random and creative. It allows the model to take more risks and choose less probable words, leading to more varied and imaginative responses. we can see this effect with `llm_creative` (temperature=2.0) in the notebook, where it generated different definitions for 'Idea' across multiple runs.

```python
from langchain_google_genai import ChatGoogleGenerativeAI

# Model A: The "Accountant" (Precision)
llm_focused = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.0)

# Model B: The "Poet" (Creativity)
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=2.0)
```

```python
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
Run 2: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
```

```python
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a thought, concept, or mental impression formed in the mind.
Run 2: An idea is a thought, concept, or mental impression formed in the mind.
```

```python
print("--- CREATIVE (Temp=2) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=2) ---
Run 1: An idea is a thought, concept, or mental image that exists in the mind as a potential plan, understanding, or solution.
Run 2: An idea is a thought, concept, or mental impression that exists in the mind, often as a result of understanding, imagination, or planning.
```

- SystemMessage controls the AI's persona and tone. By defining the AI as a "rude teenager," the model responded with slang and an informal style, showing that explicit role definition can significantly alter the LLM's output personality. This is crucial for guiding the model's behavior beyond a simple prompt.
- ChatPromptTemplate is used for structured and safe prompt creation. It showed how to define a template with placeholders (e.g., {input_language}, {text}) that clearly specifies the required input variables. This approach prevents issues that can arise from direct string concatenation, making prompts more robust and easier to manage, especially when handling dynamic user inputs.
- StrOutputParser is needed for extracting clean, usable text from the LLM's raw output. The `llm.invoke()` method returns a more complex AIMessage object, but often only the content string is needed. The StrOutputParser efficiently strips away the additional metadata, providing just the plain text content, which simplifies downstream processing in applications.

```
from langchain_core.messages import SystemMessage, HumanMessage

# Scenario: Make the AI rude.
messages = [
    SystemMessage(content="You are a rude teenager. You use slang and don't care about grammar."),
    HumanMessage(content="What is the capital of France?")
]

response = llm.invoke(messages)
print(response.content)
```

```
Ugh, Paris. Obvs. Like, everyone knows that. Duh.
```

```
from langchain_core.messages import SystemMessage, HumanMessage

# Scenario: Make the AI rude.
messages = [
    SystemMessage(content="You are a chef, all you about is food."),
    HumanMessage(content="What is the capital of France?")
]

response = llm.invoke(messages)
print(response.content)
```

```
Ah, Paris! The capital of France, and oh, what a capital it is for the palate! When I think of Paris, my mind immediately drifts to the warm, butter
```

Changing the AI’s tone to sound like a chef.

The main takeaway is how to build efficient and readable AI application pipelines. Here's what happened:

1. **The Problem (Spaghetti Code):** The notebook first demonstrated the "manual way" of connecting different components (like a prompt, an LLM, and an output parser). This involved calling each step sequentially, passing the output of one to the input of the next. It's verbose and can become cumbersome as the pipeline grows.
2. **The Solution (LCEL):** The LangChain Expression Language (LCEL) was then introduced as a much cleaner and more powerful way to achieve the same result. LCEL uses the **pipe operator (`|`)** to chain components together. This means the output of one component automatically becomes the input for the next, just like pipes in a Unix command line.

    - For example, we saw how `template | llm | parser` creates a complete chain in a single, elegant line.
3. **Why LCEL is Critical (Composability):** The main benefit of LCEL is **composability**. This means:
    - **Easier Modification:** If you want to swap out the LLM for a different one, you only change one variable in your chain definition, not multiple `llm.invoke()` calls throughout your code.
    - **Easier Extension:** If you need to add a new step (e.g., a data transformation, a spellchecker) to your pipeline, you can simply insert it into the chain using another `|` operator, without rewriting the entire flow.

Essentially, Part 1c taught you to build AI application logic like a LEGO set: snapping together different pieces (prompts, models, parsers) to create a powerful and flexible workflow, rather than manually connecting each piece with individual wires.

Assignment:

This assignment demonstrates the use of the LangChain Expression Language (LCEL) to build a complete AI pipeline in a clear and organized manner. The chain (`movie_template | llm | parser`) was constructed in a single line, showing how LCEL allows different components to be connected in a modular and readable way. This makes the workflow easy to understand, maintain, and extend for future tasks.

The implementation also correctly uses datetime.datetime.now().year to dynamically obtain the current year. This ensures that the calculation of “years ago” is always accurate and does not rely on hardcoded values, making the solution more robust and adaptable over time.

The prompt was carefully designed to guide the language model to produce output in a specific format: ‘YYYY (Years Ago: X)’. This clear instruction helped the model generate consistent and structured responses, which reduced the need for complex post processing.

The gemini-2.5-flash model successfully extracted the release year of the given movies, such as “Interstellar” and “The Matrix”, and performed the simple arithmetic required to calculate how many years ago each movie was released. This shows the model’s ability to handle both information retrieval and basic reasoning based on the prompt.

Finally, the StrOutputParser ensured that the final output contained only the clean, formatted text without any additional metadata. Overall, the assignment effectively demonstrates how LCEL, prompt engineering, and output parsing can be combined to create a simple and reliable AI powered pipeline.

```python
import datetime

# Get the current year dynamically
current_year = datetime.datetime.now().year

# Create the prompt template
movie_template = ChatPromptTemplate.from_template(
    "What is the release year of {movie_name}? And how many years ago was that, assuming the current year is {current_year}? "
    "Just provide the release year and the years ago in the format 'YYYY (Years Ago: X)'."
)

# Define the LCEL chain
movie_chain = movie_template | llm | parser

# Invoke the chain with a movie name
movie_name = "Interstellar"
result = movie_chain.invoke({"movie_name": movie_name, "current_year": current_year})

print(f"Movie: {movie_name}")
print(f"Result: {result}")
```

```
Movie: Interstellar
Result: 2014 (Years Ago: 12)
```

```python
movie_name = "The Matrix"
result = movie_chain.invoke({"movie_name": movie_name, "current_year": current_year})

print(f"Movie: {movie_name}")
print(f"Result: {result}")
```

```
Movie: The Matrix
Result: 1999 (Years Ago: 27)
```

Part 2: Prompt Engineering

Class notes:

Q.Where is COT and Graph of thought used?

Ans. In **prompt engineering**, **Chain of Thought (CoT)** is used to guide models to solve problems step-by-step by explicitly asking for reasoning (e.g., “think step by step”).

It improves accuracy in tasks like math, coding, and logical reasoning.

Graph of Thoughts (GoT) extends this by allowing multiple reasoning paths instead of a single linear chain.

It helps explore alternative solutions and compare them before selecting the best one.

Both techniques make AI responses more reliable, structured, and effective for complex prompts.

Part 2a:

- **Stochasticity (Randomness):** Explains that LLMs predict the 'next token' based on probabilities, leading to varied outputs. It highlights `temperature` as a control for this randomness.
- **CO-STAR Framework:** Presents a structured approach for crafting effective prompts, emphasizing **C**ontext, **O**bjective, **S**tyle, **T**one, **A**udience, and **R**esponse Format to guide the model precisely.

- **Hallucination vs. Creativity:** Clarifies that when specific facts are absent, the model 'fills in the gaps' based on learned patterns, which is a feature of its generative nature, not a flaw.
- **Ambiguity is the Enemy:** Stresses the importance of clear and specific instructions in prompts, as any unstated information will be inferred by the model, potentially leading to unintended outputs.

Part 2b:

- **In-Context Learning:** Describes how LLMs can learn patterns and behaviors directly from examples provided within the prompt itself, utilizing their attention mechanism.
- **Zero-Shot Prompting:** Illustrates how models respond without any specific examples, relying solely on their pre-trained knowledge.
- **Few-Shot Prompting:** Demonstrates the effectiveness of providing a few examples to guide the model towards a desired output format, style, or task, showing how the model adapts its responses.
- **Data Quality:** Emphasizes that the quality of examples provided in few-shot prompting is crucial; bad examples lead to bad patterns.

Part 2c:

- **Engineering vs. Training:** Distinguishes between **Hard Prompts** (prompt engineering, modifying input text, cheap and fast for general tasks) and **Soft Prompts** (fine-tuning, modifying model weights, expensive and slow for domain-specific behavior).
- **Dynamic Few-Shotting:** Introduces the concept of programmatically selecting and injecting the most relevant examples into a prompt (e.g., using semantic search on an example database), especially when dealing with many examples or limited context windows.
- **Prompt Template Structure:** Highlights the benefit of using tools like `FewShotChatMessagePromptTemplate` for clear separation of instructions and examples, improving the model's ability to focus and understand.

Part 3: Advanced Prompting

Part 3a:

Concept: Latent Reasoning

CoT works by leveraging the LLM's nature as a “Next Token Predictor”. By forcing the model to generate intermediate steps, these steps become part of the context, allowing the model to attend to them and refine its internal reasoning process before generating the final output. Essentially, “writing is thinking” for LLMs.

Experiment: A Tricky Math Problem

A problem like:
“Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many does he have now?”
is used to show:

1. **Standard Prompt:** Often leads to incorrect or less robust reasoning (e.g., just adding 5 and 2).
2. **CoT Prompt:** By adding the phrase “Let’s think step by step.”, the model breaks down the problem, performs multiplication first, and then addition, leading to the correct answer (11).

Part 3b:

Introduction

While CoT is linear, complex reasoning often requires nonlinear approaches. This section introduces ToT and GoT for more sophisticated problem-solving.

Tree of Thoughts (ToT)

ToT explores multiple potential reasoning paths (branches) before making a decision. The analogy used is a chess player considering several moves.

Implementation:

1. **Branch Generation:** Generate multiple distinct solutions (e.g., 3 creative solutions for “How can I get my 5-year-old to eat vegetables?”).
2. **Judging/Selection:** A “Judge” (another prompt to the LLM, acting as a child psychologist) evaluates these branches and selects the most appropriate or sustainable solution, explaining the rationale.

Graph of Thoughts (GoT)

GoT is a more advanced framework where reasoning is structured as a network. Information can diverge, process independently, and then aggregate.

Analogy: A writer's room.

Steps:

1. **Split/Generate:** Create multiple independent ideas (e.g., movie plots in Sci-Fi, Romance, Horror).
2. **Aggregate:** Combine the best elements from all ideas into a "Master Plot".
3. **Refine:** Polish the aggregated output.

4. Summary & Comparison Table

Method	Structure	Best For...	Cost/Latency
Simple Prompt	Input -> Output	Simple facts, summaries	⭐ Low
CoT (Chain)	Input -> Steps -> Output	Math, Logic, Debugging	⭐⭐ Med
ToT (Tree)	Input -> 3x Branches -> Select -> Output	Strategic decisions, Brainstorming	⭐⭐⭐ High
GoT (Graph)	Input -> Branch -> Mix/Aggregate -> Output	Creative Writing, Research Synthesis	⭐⭐⭐⭐ V. High

Recommendation: Start with CoT. Only use ToT/GoT if CoT fails.

Part 4: RAG and Vector Stores

1. **Embeddings**: Explains how text is converted into numerical vectors for computers to understand similarity.
    - Demonstrates how to generate embeddings using `HuggingFaceEmbeddings` (MiniLM).
    - Illustrates the concept of dimensionality in vectors.
    - Explains Cosine Similarity as a method to measure the closeness between vectors, with practical examples comparing 'Cat', 'Dog', and 'Car'.
2. **Naïve RAG Pipeline**: Introduces the basic architecture of RAG.
    - Sets up a simple knowledge base using `langchain_core.documents.Document`.
    - Demonstrates Indexing using `FAISS` (Facebook AI Similarity Search) to store document embeddings.
    - Builds a basic RAG chain using `langchain_core.prompts.ChatPromptTemplate`, `langchain_core.output_parsers.StrOutputParser`, and `langchain_core.runnables.RunnablePassthrough` with a `ChatGoogleGenerativeAI` LLM.

    - Shows how the RAG chain retrieves relevant context and generates an answer.
3. **Deep Dive into Indexing Algorithms**: Explores advanced FAISS indexing techniques to handle large-scale vector searches efficiently.
    - Discusses the "Trade-off Triangle" of Speed, Accuracy, and Memory.
    - Introduces and demonstrates different FAISS index types with mock data:
        - Flat Index (`IndexFlatL2`): Brute-force search for 100% accuracy but slow.
        - IVF (Inverted File Index `IndexIVFFlat`): Uses clustering for faster search by partitioning data.
        - HNSW (Hierarchical Navigable Small World `IndexHNSWFlat`): Builds a graph for extremely fast retrieval, using a 'six degrees of separation' concept.
        - PQ (Product Quantization `IndexPQ`): A compression technique to reduce memory usage, trading some accuracy for efficiency.

1. What is a Vector DB? What is RAG?

A vector database stores data as embeddings (numerical vectors) for fast similarity search.
RAG (Retrieval-Augmented Generation) combines retrieval from external data with LLM generation.
It helps models give more accurate, context-aware answers.

2. RAG has 4 parts

Ingestion: Collect and load data into the system.
Chunking & Embeddings: Split text into pieces and convert them into vectors.
Storage & Retrieval: Store in a vector DB and retrieve relevant chunks during queries.

3. Challenges in RAG

Too much noise: Irrelevant chunks may be retrieved.
Poor chunking or embeddings can reduce accuracy.
Latency and scaling issues with large datasets.

4. What is Quantization?

Quantization reduces model size by lowering precision (e.g., float32 → int8).
It speeds up inference and reduces memory usage.
But it may slightly reduce model accuracy.

Assignment MOE

Objective

To build an intelligent routing system that classifies user intent and directs the query to the most appropriate expert.
This is achieved using the same LLM (**llama-3.1-8b-instant via Groq**) with different system prompts and temperature settings.

Architecture Overview

Flow:

User Query
→ Router LLM (classifies intent)
→ Expert System Prompt / Function Call
→ Expert Response

Categories: technical / billing / general / crypto

Components and Implementation Details

1. Setup and API Key

- Dependencies: `groq`, `python-dotenv`
- API key is securely loaded from `.env` or via `getpass`
- Groq client is initialized using the API key
- Base model used: **llama-3.1-8b-instant**

2. Expert Configurations

- Defined using a `MODEL_CONFIG` dictionary
- Each expert includes:
    - **System Prompt** → defines role and behavior
    - **Temperature** → controls response style

Temperature settings:

- 0.7 → technical & billing (structured responses)
- 0.8 → general (more conversational)

3. The Router

Function: `route_prompt(user_input: str) -> str`

Purpose: Classifies user queries into categories

Key Points:

- Uses a constrained system prompt
- Returns only one category
- Uses **temperature = 0** for deterministic output
- Acts as a **Zero-Shot Classifier**
- Defaults to “general” if classification fails

4. The Orchestrator

Function: `process_request(user_input: str) -> str`

Purpose: Manages full request flow

Steps:

1. Calls router to classify query
2. Fetches corresponding expert configuration
3. Sends query with expert system prompt to LLM
4. Returns generated response

5. Bonus: Tool-Use Expert

Introduces function-based expert handling instead of LLM response.

Key Components:

- `mock_fetch_crypto_price()` → simulates crypto price retrieval
- `route_prompt_v2()` → adds “crypto” category
- `process_request_v2()` → enhanced orchestrator

Flow:

1. Classify query
2. If crypto:
    - Extract coin name using LLM
    - Call function directly
3. Else:
    - Use standard LLM expert

Key Takeaways

- MoE enables specialization using prompt engineering
- Router with temperature = 0 ensures consistent classification
- Experts use different temperatures for varied response styles
- Orchestrator connects all components into a pipeline
- Tool-use experts simulate real-world API integration

```python
# --- Test 1: Technical Query ---
print("=" * 60)
result = process_request("My Python script is throwing an IndexError on line 5.")
print(f"\n💬 Expert Response:\n{result}")
print("=" * 60)
```

```
============================================================
📩 User Input : My Python script is throwing an IndexError on line 5.
🔀 Routed to  : [TECHNICAL EXPERT]

💬 Expert Response:
I'd be happy to help you troubleshoot the issue.

To start, could you please provide more details about the error message, such as:

1. The full error message, including any stack traces or error codes.
2. The relevant code snippet from lines 1-10, especially around line 5.
3. The input data or environment that's causing the error.

This information will help me identify the root cause of the issue and provide a more accurate solution.

In general, the `IndexError` occurs when you're trying to access an element at an index that doesn't exist in the list or sequence.

Here's a general example of an `IndexError`:

```python
# Example code with IndexError
my_list = [1, 2, 3]
print(my_list[3])  # Raises IndexError: list index out of range
```

Let's get started with your specific issue.
==
```

```python
# --- Test 2: Billing Query ---
print("=" * 60)
result = process_request("I was charged twice for my subscription this month.")
print(f"\n💬 Expert Response:\n{result}")
print("=" * 60)
```

```
============================================================
📩 User Input : I was charged twice for my subscription this month.
🔀 Routed to  : [BILLING EXPERT]

💬 Expert Response:
I'm so sorry to hear that you've been charged twice for your subscription. That can be really frustrating and stressful.

Firstly, let me reassure you that I'm here to help and we'll get this issue sorted out as soon as possible. Can you please provide me with your acco

Once I have that information, I'll check on our end to see what happened and work on reversing the duplicate charge as soon as possible. You can exp

In the meantime, if you'd like to avoid any further charges for the current month, I can also assist you with updating your subscription to cancel o
============================================================
```